

USER GUIDE REPORT FOR LOAN STATUS PREDICTION PIPELINE

Author: Onyinye Ezike
Student No: 24156343
Module: Computing For AI
Module code: CMP6221
Work: Assessment 2
Lecturer: Yekta Said Can
Program: MSc Artificial Intelligence with Professional Placement

Onyinye Ezike
24156343 onyinye.ezike@mail.bcu.ac.uk

LOAN STATUS PREDICTION – A COMPREHENSIVE USER GUIDE

[\(Link To Video Version of this User Guide\)](#)

Table of Contents

1. [Introduction](#)
 - 1.1. Overview
 - 1.2. Objective
2. [Case Study Description](#)
 - 2.1. Context
 - 2.2. Data Summary
3. [Program Design](#)
 - 3.1. Features and Functionality
 - 3.2. Class and Function Description
 - 3.2.1. DataLoader Class
 - 3.2.2. Data Preprocessor Class
 - 3.2.3. Data Visualizer Class
 - 3.2.4. LoanStatusPredictor Class
 - 3.2.5. ModelEvaluator Class
4. [Algorithms and Model Selection](#)
 - 4.1. Machine Learning Models/Algorithms
 - 4.2. Model Evaluation
 - 4.3. Main Function
 - 4.4. Connectivity and IO
5. [Program Usage Guide](#)
 - 5.1. System Requirements
 - 5.2. Setup Instructions
 - 5.3. Program Operation
 - 5.4. Expected Outputs/Output Interpretation
6. [Result Interpretation](#)
 - 6.1. Key Insights
 - 6.2. Model Comparison
7. [Conclusion](#)
8. [Appendix](#)
 - 8.1. UML Diagrams
 - 8.2. Sample Output Screenshots
9. [Reference](#)

1. Introduction

1.1 Overview

This document provides a comprehensive user guide for the Loan Status Prediction program, detailing its design, functionality, and usage instructions. This program leverages machine learning to predict the status of loan applications based on historical data. It incorporates data loading, preprocessing, visualization, model training, and evaluation functionalities. The Loan Status Prediction aims to assist commercial banks in determining the eligibility of loan applicants by predicting the approval status (Approved or Rejected) of their loan applications. This program utilizes machine learning to analyze applicant's data, including demographic information, financial status, and credit history, to generate predictions. By automating the decision-making process, this pipeline can significantly simplify loan approval procedures. This program was fully implemented using Object Oriented Principles (OOP).

1.2 Objective

This program report serves as a user guide and project documentation for the Loan Status Application Prediction program, providing a breakdown of the program design, the data processing techniques, and instructions on how to operate and use the application effectively.

2. Case Study Description

2.1 Context

The loan prediction program was developed in response to a case study where financial institutions required a solution to automate the loan approval process. Given a set of historical loan data, this program predicts the outcome of new loan applications based on various features, such as applicant income, credit history, loan amount, and property area.

2.2 Data Summary

The dataset used for this project includes:

- **Loan applicant details:** gender, marital status, dependents, education, self-employment status, and property area.
- **Financial data:** applicant income, co-applicant income, loan amount, and loan term.
- **Credit history:** credit score indicating past credit reliability.
- **Target Variable:** loan status, indicating whether a loan application was approved or rejected in historical data.

3. Program Design

3.1 Features and Functionality

The program consists of multiple Python classes, each serving a unique role in data loading, preprocessing, visualization, model training, and evaluation. Key functionalities include:

- **Data Loading:** Efficiently loads loan application data from various sources.
- **Data Preprocessing:** Handles missing values, encodes categorical variables, and scales numerical features for optimal model performance.
- **Exploratory Data Analysis (EDA):** Visualizes data distributions and feature correlations to gain insights.
- **Model Training:** Trains multiple machine learning models, including Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and XGBoost.
- **Model Evaluation:** Evaluates trained models using metrics such as accuracy, classification reports, and confusion matrices.
- **Hyperparameter Tuning:** Optimizes select models (Random Forest, SVM) using GridSearchCV for enhanced accuracy.

3.2 Class and Function Descriptions

3.2.1 DataLoader Class

- **Purpose:** Load the dataset => Data loaded with shape: (381, 12)
- **Functions:**
 - `__init__(self, file_path)`: Initializes with the data file path.
 - `load_data(self)`: Loads data from the specified path, removes the Loan_ID column to allow some initial data visualization.

3.2.2 DataPreprocessor Class

- **Purpose:** Prepare data for model training by handling missing values, encoding categorical variables, scaling numerical features, and ensuring no duplicates.
- **Functions:**
 - `__init__(self, df)`: Initializes with the DataFrame.
 - `handle_missing_values(self)`: Addresses missing values in both categorical and numerical features using mode and mean imputation.
 - `encode_categorical_variables(self)`: Encodes categorical variables into numerical representations using Label Encoding.

- `scale_features(self)` : Scales numerical features using StandardScaler.
- `check_and_remove_duplicates(self)` : Detects and removes duplicate rows.
- `check_for_nan(self)` : Verifies the absence of NaN values after preprocessing.
- `preprocess(self)` : Executes all preprocessing steps sequentially.

3.2.3 DataVisualizer Class

- **Purpose:** Visualize key features of the dataset to assist in data exploration and analysis.
- **Functions:**
 - `__init__(self, df)` : Initializes with the DataFrame.
 - `plot_distributions(self)` : Visualizes the distributions of numerical and categorical features using histograms and countplots.
 - `correlation_matrix(self)` : Generates a correlation heatmap to display feature relationships

3.2.4 LoanStatusPredictor Class

- **Purpose:** Train machine learning models to predict loan approval status.
- **Functions:**
 - `__init__(self, X_train, y_train)` : Initializes with training data.
 - `logistic_regression(self)` : Trains a Logistic Regression model.
 - `decision_tree(self)` : Trains a Decision Tree Classifier.
 - `random_forest(self)` : Trains a Random Forest Classifier with hyperparameter tuning using GridSearchCV.
 - `svm(self)` : Trains a Support Vector Machine (SVM) model with hyperparameter tuning using GridSearchCV.
 - `knn(self)` : Trains a K-Nearest Neighbors (KNN) model.
 - `xgboost(self)` : Trains an XGBoost Classifier.

3.2.5 ModelEvaluator Class

- **Purpose:** Evaluate and report on the performance of each trained model.
- **Function:**
 - `__init__(self, X_test, y_test)` : Initializes with testing data.
 - `evaluate(self, model)` : Evaluates a trained model using accuracy, classification report, and confusion matrix.

4. Algorithms and Model Selection

4.1 Machine Learning Models/Algorithms

The program employs six machine learning algorithms:

1. **Logistic Regression:** Suitable for binary classification problems; calculates the probability of loan approval.
2. **Decision Tree:** A non-parametric supervised learning model ideal for decision-based predictions.
3. **Random Forest:** An ensemble model that averages multiple decision trees to improve prediction accuracy.
4. **Support Vector Machine (SVM):** Effective in high-dimensional spaces, used for classification with linear or non-linear kernels.
5. **K-Nearest Neighbors (KNN):** Classifies based on the proximity of an instance to its neighbors.
6. **XGBoost:** A gradient-boosted decision tree algorithm that is effective for structured data.

4.2 Model Evaluation

Each model is evaluated using:

- **Accuracy:** Proportion of correct predictions.
- **Classification Report:** Precision, recall, and F1 score for both classes (Approved/Rejected).
- **Confusion Matrix:** Shows true positives, true negatives, false positives, and false negatives.

4.2 Main Function

main(): The principal function that orchestrates the entire program flow, including data loading, preprocessing, visualization, model training, and evaluation.

4.3 Connectivity and IO

- **Data Loading:** The program can load data from local files or URLs.
- **Output:** The program outputs the evaluation metrics for each trained model, including accuracy, classification report, and confusion matrix.

5. Program Usage Guide

5.1 System Requirements

- **Python:** Version 3.8 or above
- **Libraries:** pandas, matplotlib, seaborn, scikit-learn, xgboost

5.2 Setup Instructions

1. **Install dependencies:** `pip install pandas matplotlib seaborn scikit-learn xgboost`
2. **Data Preparation:** Prepare your loan application data in a CSV format. The data should have features relevant to loan applications, such as applicant income, loan amount, credit history, etc.
3. **Script Modification:**
 - Open the `loan_status_application_prediction.py` script in a text editor or IDE.
 - Update the `file_path` variable in the `main()` function to point to your data file.
 - (Optional) Modify the models dictionary in the `main()` function to add or remove models for training.
4. **Execution:**
 - Open a terminal or command prompt in the project directory.
 - Run the script with: `python loan_status_application_prediction.py`

5.3 Program Operation

1. **Data Loading:** The `DataLoader` class loads and cleans the dataset, dropping irrelevant columns.
2. **Data Preprocessing:**
 - `DataPreprocessor` handles missing values, encoding, and scaling.
 - Duplicate records are removed to ensure data quality.
3. **Data Visualization:**
 - `DataVisualizer` produces plots to help understand feature distributions and relationships.
 - Run `visualizer.plot_distributions()` and `visualizer.correlation_matrix()` for visual insights.
4. **Model Training and Evaluation:**
 - The `LoanStatusPredictor` class trains each machine learning model.
 - The `ModelEvaluator` class evaluates each model's accuracy, printing a summary of performance metrics.

5.3 Expected Outputs/Output Interpretation

The program will print the evaluation metrics for each trained model to the console. Analyze these metrics to understand the performance of each model and select the best one for your needs. After running the program, the user will see:

- **Data summaries:** A summary of missing values, unique values, and duplicates.
- **Visualization output:** Various plots for data exploration.
- **Evaluation metrics:** Accuracy, classification report, and confusion matrix for each model.

6. Results Interpretation

6.1 Key Insights

The program provides insights into which features are most predictive of loan status. The program has visualizations that can help in understanding the distributions of features and the relationships between them. For instance, Credit_History often has a strong impact on loan approval predictions.

6.2 Model Comparison

Models are compared based on accuracy and other metrics to determine the best-performing model for the given dataset.

Model Performance Summary

Model	Accuracy Score
Logistic Regression	84.42%
Decision Tree	70.13%
Random Forest	84.42%
Support Vector Machine	84.42%
K-Nearest Neighbors	68.83%
XGBoost	84.42%

Based on these results, **Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost** models all achieved the highest accuracy score of **84.42%**. Given that these four models performed equally well in terms of accuracy, additional criteria such as model interpretability, computational efficiency, or robustness may be considered to make a final choice among them.

7. Conclusion

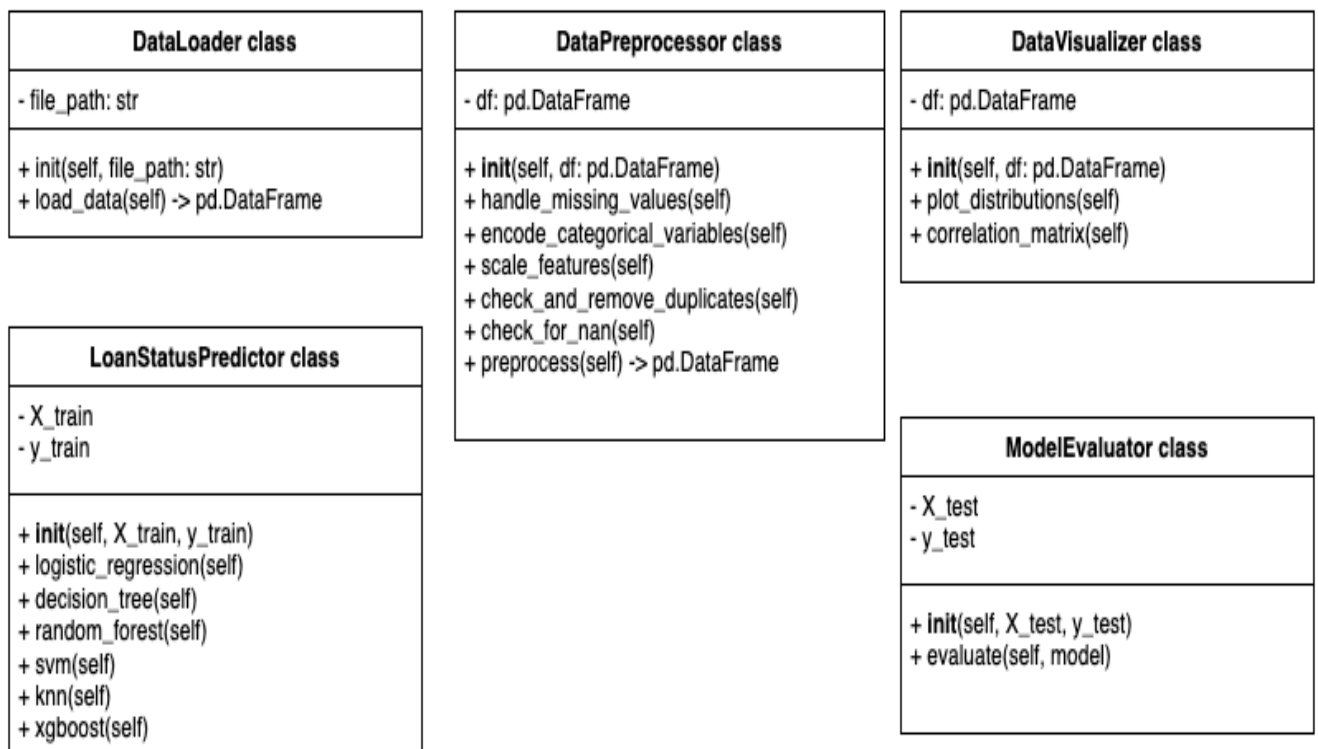
The Loan Status Application Prediction project offers an efficient solution for automating loan application assessments. The program is modular and well-organized, making it easy to understand and extend. The user-friendly, modular design enables easy updates to data preprocessing and model selection, allowing flexibility and scalability.

8. Appendix

8.1 UML Diagrams

The below UML was created using Draw.io. [This is the Draw.io Link of the UML Diagram.](#)

UML Diagrams For The Classes



8.2 Sample Output Screenshots

- Loan Data Sample

loan_data

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Urban	Y
LP001013	Male	Yes	0	Not Graduate	No	2333	1516.0	95.0	360.0	1.0	Urban	Y
LP001024	Male	Yes	2	Graduate	No	3200	700.0	70.0	360.0	1.0	Urban	Y
LP001027	Male	Yes	2	Graduate		2500	1840.0	109.0	360.0	1.0	Urban	Y
LP001029	Male	No	0	Graduate	No	1853	2840.0	114.0	360.0	1.0	Rural	N

- First Five Rows of Loan Data set After Encoding

```

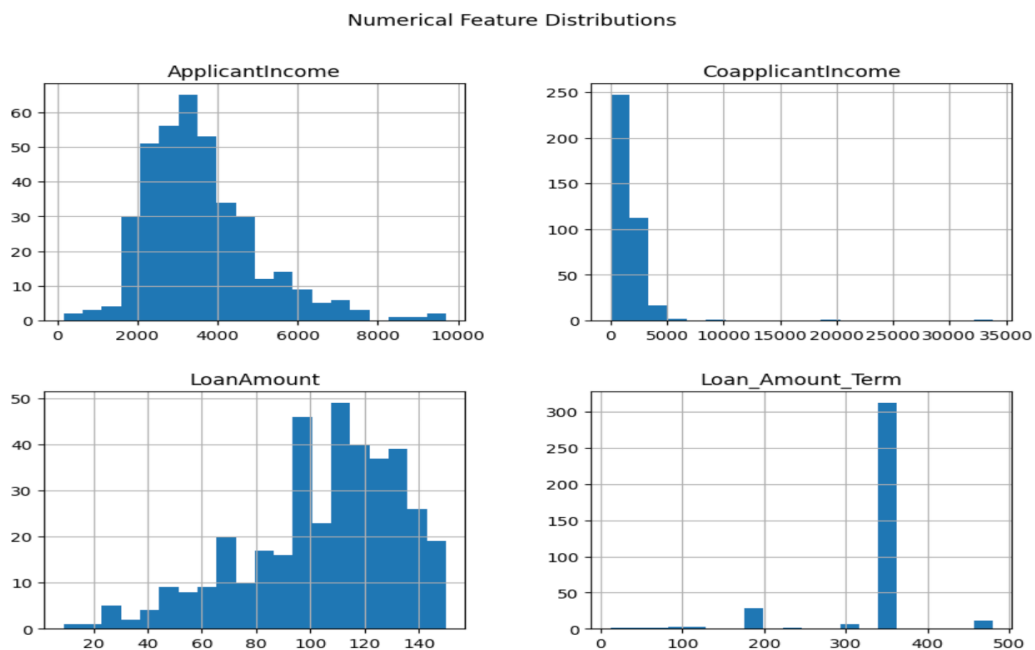
First Five Rows After Encoding:
Gender    Married    Dependents    Education    Self_Employed    ApplicantIncome    \
0         1         1         1         0         0         0.707469
1         1         1         0         0         0         -0.408932
2         1         1         0         0         1         -0.703019
3         1         0         0         0         0         1.706799
4         1         1         0         1         0         -0.879330

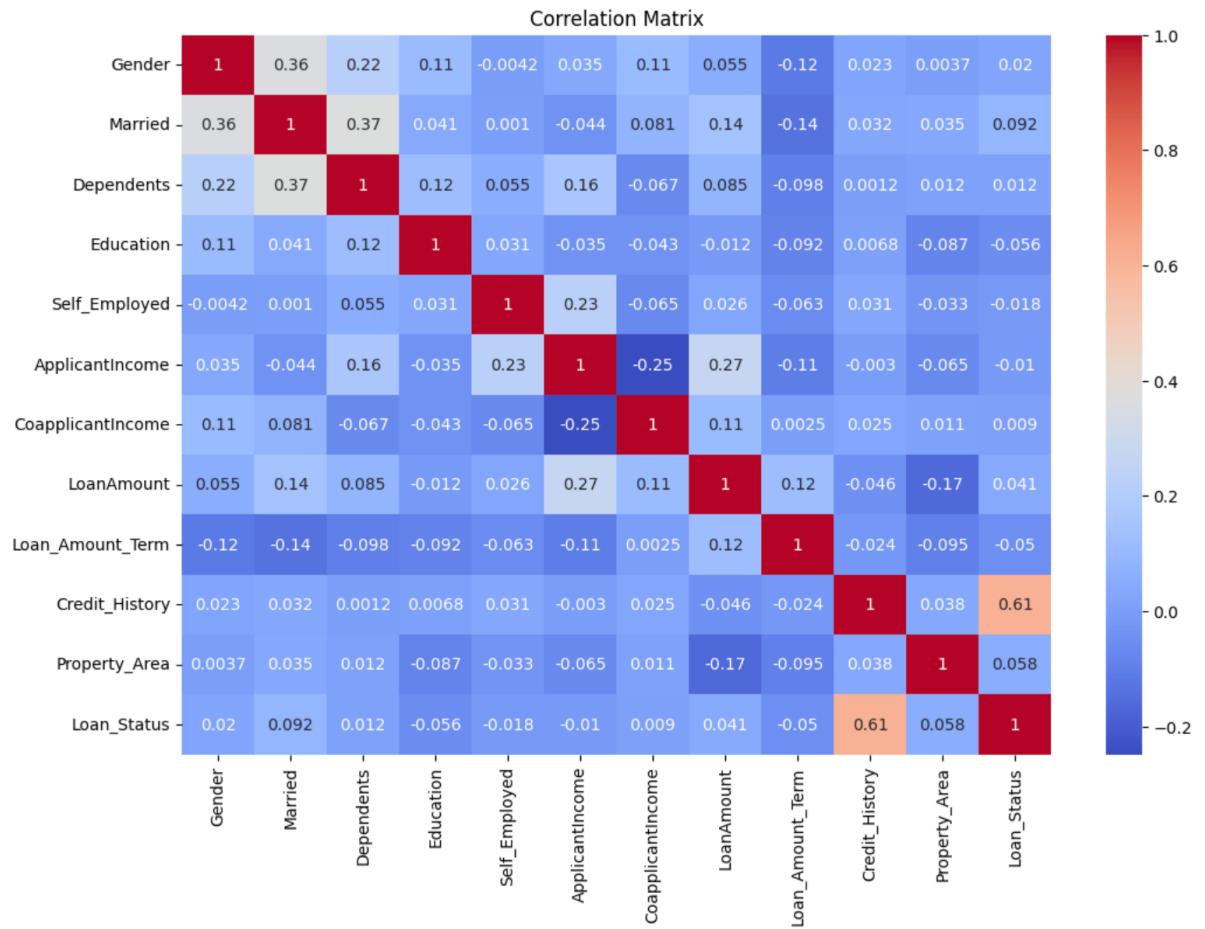
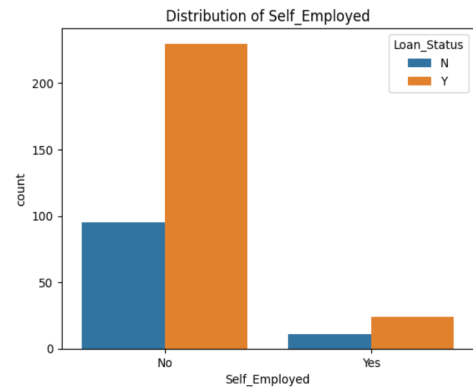
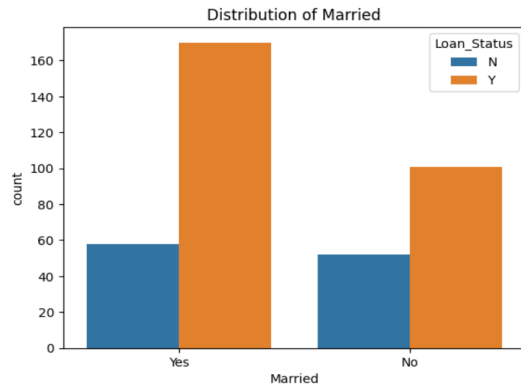
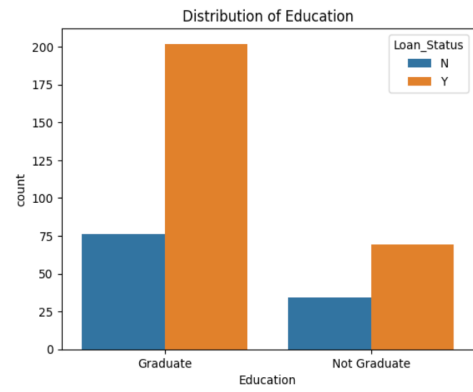
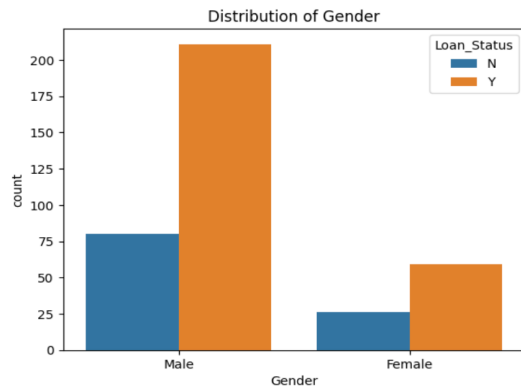
CoapplicantIncome    LoanAmount    Loan_Amount_Term    Credit_History    \
0         0.098695         0.812575         0.275147         1.0
1        -0.546371        -1.376596         0.275147         1.0
2         0.462294         0.530102         0.275147         1.0
3        -0.546371         1.271595         0.275147         1.0
4         0.102118        -0.352629         0.275147         1.0

Property_Area    Loan_Status
0         0         0
1         2         1
2         2         1
3         2         1
4         2         1

```

- Data Visualizations





- **Model Accuracy Results and Confusion Matrices**



Evaluating Logistic Regression Model:

Accuracy: 0.8441558441558441

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.48	0.65	23
1	0.82	1.00	0.90	54
accuracy			0.84	77
macro avg	0.91	0.74	0.77	77
weighted avg	0.87	0.84	0.82	77

Confusion Matrix:

```
[[11 12]
 [ 0 54]]
```

Evaluating Decision Tree Model:

Accuracy: 0.7012987012987013

Classification Report:

	precision	recall	f1-score	support
0	0.50	0.48	0.49	23
1	0.78	0.80	0.79	54
accuracy			0.70	77
macro avg	0.64	0.64	0.64	77
weighted avg	0.70	0.70	0.70	77

Confusion Matrix:

```
[[11 12]
 [11 43]]
```

Evaluating Random Forest Model:

Accuracy: 0.8441558441558441

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.48	0.65	23
1	0.82	1.00	0.90	54
accuracy			0.84	77
macro avg	0.91	0.74	0.77	77
weighted avg	0.87	0.84	0.82	77

Confusion Matrix:

```
[[11 12]
 [ 0 54]]
```

Evaluating SVM Model:

Accuracy: 0.8441558441558441

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.48	0.65	23
1	0.82	1.00	0.90	54
accuracy			0.84	77
macro avg	0.91	0.74	0.77	77
weighted avg	0.87	0.84	0.82	77

Confusion Matrix:

```
[[11 12]
 [ 0 54]]
```

```

Evaluating KNN Model:
Accuracy: 0.6883116883116883
Classification Report:
              precision    recall  f1-score   support

     0       0.40      0.09      0.14         23
     1       0.71      0.94      0.81         54

 accuracy          0.69         77
 macro avg          0.55         77
 weighted avg       0.62         77

```

```

Confusion Matrix:
[[ 2 21]
 [ 3 51]]

```

```

Evaluating XGBoost Model:
Accuracy: 0.8441558441558441
Classification Report:
              precision    recall  f1-score   support

     0       0.82      0.61      0.70         23
     1       0.85      0.94      0.89         54

 accuracy          0.84         77
 macro avg          0.84         77
 weighted avg       0.84         77

```

```

Confusion Matrix:
[[14  9]
 [ 3 51]]

```

References

1. [Viswanatha, V., & Ramachandra, A. C. \(2023\). Prediction of Loan Approval in Banks using Machine Learning Approach. \[Journal Name, Volume\(Issue\), Pages\].](#)
2. Pandas Documentation. (n.d.). Retrieved from <https://pandas.pydata.org/docs/>
3. Matplotlib Documentation. (n.d.). Retrieved from <https://matplotlib.org/>
4. Seaborn Documentation. (n.d.). Retrieved from <https://seaborn.pydata.org/>
5. Scikit-learn Documentation. (n.d.). Retrieved from <https://scikit-learn.org/stable/>
6. XGBoost Documentation. (n.d.). Retrieved from <https://xgboost.readthedocs.io/en/stable/>
7. Draw.io from <https://app.diagrams.net/>